

INFORME DE AVANCE N°2: DISEÑO Y DESARROLLO FUNCIONAL INTEGRADO

Asignatura: Taller Aplicado de Programación (TPY 1101)

Proyecto: Menú Bites

Grupo: Grupo 2

Versión: 2.8.0

Fecha: 19 de Mayo de 2026

1. RESUMEN EJECUTIVO Y EQUIPO ACADÉMICO

1.1 Resumen Ejecutivo

Menú Bites es una plataforma de software como servicio (SaaS) multi-tenant orientada a la industria gastronómica que permite a múltiples restaurantes operar de forma digital e independiente sobre una infraestructura tecnológica compartida. La solución aborda la creciente necesidad de digitalización en el sector gastronómico, ofreciendo una alternativa accesible frente a sistemas tradicionales costosos o infraestructuras separadas por establecimiento.

La plataforma entrega a cada restaurante su propio entorno digital aislado de manera lógica: menú interactivo accesible mediante código QR o NFC, gestión de pedidos en tiempo real, panel de administración para configurar productos, mesas y personal, y monitores de despacho tipo Kanban para el seguimiento de la preparación (Kitchen KDS y Bar Dashboard). El aislamiento de datos entre restaurantes se garantiza mediante Row Level Security (RLS) a nivel de base de datos en PostgreSQL/Supabase, evitando el acceso cruzado de información.

El modelo de negocio se basa en una suscripción mensual por restaurante gestionada por un super administrador global, quien controla el estado de cada tenant desde un panel centralizado. Técnicamente, la solución se desarrolla como un monorepo con múltiples aplicaciones web optimizadas y una aplicación móvil responsiva, compartiendo tipos, validaciones y clientes de API desde un paquete común que reduce la duplicación de código y asegura la consistencia técnica.

1.2 Estructura de Roles y Responsabilidades

El equipo de trabajo se distribuye bajo una matriz de responsabilidades que asocia perfiles de especialidad técnica con las necesidades de desarrollo del monorepo y la base de datos.

Integrante	RUT	Rol Académico	Responsabilidades Técnicas
Héctor Omar Antonio Robledo Aguilar	21.150.403-K	QA Engineer / Arquitecto de Datos	Diseño del esquema relacional (Master).
Alejandro Andrés Placencia Menares	17.070.619-6	Fullstack Developer / Diseñador UI/UX	Diseño y prototipado de interfaces financieras.
José Luis Medina Mercado	13.731.386-3	Product Owner / Project Manager	Gestión del backlog y priorización.

2. CONTEXTO, PROBLEMÁTICA Y ANÁLISIS OPERATIVO

2.1 Descripción de la Problemática

En la industria gastronómica contemporánea, especialmente en el segmento de pequeñas y medianas empresas (PYMEs), el núcleo del problema reside en la desconexión operativa y la alta dependencia de procesos manuales. A pesar de la existencia de sistemas POS (Point of Sale) tradicionales, muchos establecimientos operan mediante un flujo fragmentado donde la toma de pedidos, la comunicación con la cocina y la gestión administrativa ocurren de forma aislada.

Esta falta de integración se traduce en tres dimensiones críticas de ineficiencia:

- Latencia en el Servicio: El tiempo transcurrido desde que un cliente decide su orden hasta que esta llega a la cocina se ve penalizado por traslados físicos del personal y duplicidad de registros en comandas de papel.
- Fugas de Información y Capital: La ausencia de sincronización en tiempo real entre inventario, ventas y caja facilita errores humanos en el cobro y falta de trazabilidad en el consumo de insumos.
- Barrera de Entrada Digital: El software especializado suele implicar costos de licenciamiento por sucursal o hardware propietario, haciendo que la digitalización sea financieramente inalcanzable para emprendimientos emergentes.

2.2 Análisis Causa-Efecto

Para comprender la raíz de las ineficiencias del sistema actual sin digitalización, se ha estructurado una jerarquía de causas y sus correspondientes impactos financieros y operacionales sobre el negocio.

Dimensión	Elementos de Causa Raíz	Pr
Flujo de Información	Dependencia absoluta del soporte físico (papel) y comandas escritas a mano por el personal.	Op caj
Comunicación Interna	Traspaso asíncrono de requerimientos debido al traslado físico del garzón entre estaciones.	Cu
Control Administrativo	Falta de herramientas de monitoreo en tiempo real para la toma de decisiones por la gerencia.	Ine

2.3 Estado del Arte y Homologación

Las alternativas existentes en el mercado resuelven de manera parcial las necesidades operativas de un restaurante, pero imponen restricciones de costo o infraestructura que limitan su adopción masiva.

Criterio de Comparación	POS Tradicional	Aplicaciones de Delivered
Costo de Entrada	Alto (requiere licencias iniciales y hardware de terminal propietario).	Alto (comisiones por transacción de 30%).
Integración de Estaciones	Parcial (conexión por cable local limitada).	Inexistente (foco exclusivo en el cliente).
Aislamiento Multi-tenant	No aplica (instalaciones de base de datos individuales locales).	No (base de datos masiva en la nube local).
Independencia de Hardware	No (requiere computadores o impresoras certificadas).	No (depende de tablets o smartphones).
Infraestructura Requerida	Servidores locales e infraestructura de red interna compleja.	Dependencia absoluta de Internet.

2.4 Situación Inicial (Flujo de Proceso As-Is)

El análisis del flujo de atención tradicional revela múltiples puntos de latencia y riesgo de pérdida de información en un establecimiento que no cuenta con un sistema integrado.

Etapas	Paso Secuencial	Acción Operativa	Punto de Control
1	Llegada del Cliente	El cliente toma asiento y espera a ser atendido por el garzón disponible.	Recepción
2	Entrega de Carta	El garzón se desplaza a buscar la carta física, la lleva a la mesa y se retira.	Cartera
3	Toma de Pedido	El garzón regresa a la mesa, anota los platos seleccionados en una libreta de papel.	Errores de transcripción
4	Traslado a Cocina	El garzón camina hacia la cocina o barra y cuelga la comanda de papel en el casillero.	Pérdida de información
5	Preparación	El personal de cocina lee la comanda escrita a mano e inicia la preparación del plato.	Tiempos de espera
6	Aviso de Salida	El cocinero grita el número de mesa para avisar que el plato está listo para despacho.	Ruido de fondo
7	Entrega en Mesa	El garzón visualiza el plato listo, lo toma de la bandeja y lo lleva al cliente.	Falta de comunicación
8	Consumo	El cliente consume los alimentos y busca captar la atención del garzón para pedir la cuenta.	Pérdida de información
9	Pago y Cierre	El garzón camina a la caja, dicta los platos consumidos, el cajero calcula el total e imprime.	Errores de cálculo

3. PROPUESTA DE SOLUCIÓN Y OBJETIVOS DEL SISTEMA

3.1 Objetivo General

Desarrollar e implementar Menú Bites, una plataforma SaaS multi-tenant funcional para la gestión digital de restaurantes, que incluya menú QR/NFC, toma de pedidos en tiempo real con monitores KDS especializados de cocina y barra, y un módulo administrativo con aislamiento lógico de datos Row Level Security (RLS) en PostgreSQL/Supabase, desplegado en infraestructura de alta disponibilidad en Vercel y validado mediante pruebas automatizadas.

3.2 Alcance Técnico Integrado

El proyecto se enfoca en establecer una base de datos multitenant segura que sirva de núcleo para aplicaciones web responsivas de cara a las diferentes estaciones del restaurante.

Capa Tecnológica	Módulo de Aplicación	Mecanismo de Aislamiento y Control
Base de Datos	Supabase PostgreSQL 15	Row Level Security (RLS) a nivel de fila y triggers en PL/pgSQL.
Servicios Backend	Next.js Edge Middleware	Validación de JWT firmado, decodificación de app_metadata y resolución de slug.
Frontend Web	Admin Dashboard / KDS	Cliente unificado de Supabase con inyección del tenant context.
Frontend Móvil	Customer Portal / Waiter	Interfaz web responsiva integrada con APIs nativas del navegador (cámara, geolocalización).

3.3 Entregables del Ecosistema

El desarrollo se traduce en una suite de componentes que cubren las necesidades de todos los actores del ecosistema gastronómico.

Nº	Entregable del Proyecto	Formato de Entrega	Descripción Técnica
1	Aplicación Web Administrativa	Enlace productivo en Vercel	Panel de control para el administrador del restaurante global.
2	Kitchen Display System (KDS)	Enlace productivo en Vercel	Pantalla táctil en tiempo real para cocina que recibe pedidos.
3	Bar Dashboard	Enlace productivo en Vercel	Monitor de despacho especializado para la barra de bebidas.
4	Customer Portal	Web responsiva móvil	Interfaz pública accesible por QR o NFC para que los clientes vean el menú y realicen pedidos.
5	Base de Datos Multitenant	Instancia activa en Supabase	Esquema relacional con RLS, triggers de sincronización y procedimientos almacenados para gestión de usuarios y roles.

N°	Entregable del Proyecto	Formato de Entrega	Descripción Técnica
6	Repositorio Monorepo	Código fuente en GitHub	Estructura Turborepo que organiza las aplicaciones
7	Manual de Instalación y Operación	Formato Markdown	Guías paso a paso para la inicialización local del p

3.4 Supuestos y Restricciones

El desarrollo del proyecto está sujeto a condiciones operativas y tecnológicas que delimitan el marco de acción del equipo de trabajo.

El equipo asume que el uso de la capa gratuita de Supabase Cloud y Vercel se mantendrá estable y con límites de almacenamiento suficientes (500 MB de base de datos y 1 GB de almacenamiento de archivos) para sostener la carga de demostración académica y pruebas funcionales de extremo a extremo. Asimismo, se asume que los dispositivos móviles utilizados por los evaluadores y el equipo cuentan con soporte para navegadores modernos con acceso a la cámara y lectores NFC para el escaneo de los códigos de mesa.

En cuanto a las restricciones, el presupuesto de infraestructura cloud está limitado estrictamente a cero dólares, lo que obliga a optimizar las consultas y el peso de las imágenes almacenadas para no exceder las cuotas gratuitas. El tiempo de desarrollo está acotado al calendario académico de 18 semanas, limitando la integración de pasarelas de pago reales y forzando el uso de simuladores de transacciones en modo de prueba (Stripe Sandbox) y liquidación automatizada por caja en el local.

4. DISEÑO CONCEPTUAL Y REQUERIMIENTOS TÉCNICOS

4.1 Requerimientos Funcionales

El comportamiento del sistema está modelado a partir de un conjunto de requerimientos priorizados que garantizan la operatividad mínima de la plataforma.

ID	Módulo del Sistema	Descripción del Requerimiento
RF-01	Autenticación	El sistema debe permitir iniciar sesión mediante correo y contraseña asignando de manera automática un rol de usuario en Supabase Auth.
RF-02	Super Administrador	El panel global debe permitir la creación de nuevos tenants, la asignación de planes y la gestión de usuarios.
RF-03	Menú QR/NFC	El cliente final debe poder escanear un código en su mesa y acceder al menú interactivo del restaurante.
RF-04	División de Comandas	Al validar un pedido en salón, la base de datos debe separar automáticamente los ítems de la orden en comandas de cocina y barra.
RF-05	Monitor KDS	El personal de cocina debe contar con un panel Kanban en tiempo real con alertas audibles para cada ítem pendiente.
RF-06	Bar Dashboard	El barman debe visualizar las comandas exclusivas de bebidas en un panel dedicado y poder gestionar su estado.
RF-07	Control de Inventario	Al despacharse un pedido, el sistema debe descontar de forma automática los ingredientes de la base de datos.
RF-08	Módulo de Caja	El cajero debe poder visualizar el consumo acumulado de una mesa, calcular la propina y generar el comprobante de pago.
RF-09	Bloqueo SaaS	El sistema debe retornar un estado HTTP 402 en todos los servicios de un restaurante cuando su plan de suscripción haya expirado.
RF-10	Generación de Códigos	El administrador del restaurante debe poder crear mesas en el sistema y descargar códigos QR para cada una.

4.2 Requerimientos No Funcionales

Los atributos de calidad aseguran que la plataforma opere bajo estándares óptimos de seguridad, rendimiento y precisión.

Seguridad y Aislamiento de Datos

El aislamiento de los datos en la base de datos multi-tenant compartida se garantiza mediante Row Level Security (RLS) a nivel del motor de base de datos PostgreSQL. Esto asegura que ninguna consulta SQL pueda leer o escribir registros pertenecientes a otro restaurante, incluso si existe un fallo en la capa de aplicación web. Todas las consultas autenticadas se evalúan contrastando el `restaurant_id` del registro con el campo correspondiente codificado en la clave de firma del JWT del usuario. Las rutas que requieren privilegios de administración global (Super Admin) acceden a la base de datos mediante la clave `service_role` de Supabase, que se ejecuta con bypass de RLS de manera controlada y exclusivamente en servidores del lado de la nube de Vercel.

Confiabilidad y Disponibilidad en Tiempo Real

La plataforma delega su disponibilidad a las redes de distribución de Vercel y Supabase Cloud, garantizando un tiempo de actividad del servicio superior al 99.9%. Las actualizaciones de estado de los pedidos y comandas se transmiten a los monitores de cocina y garzones en menos de 500 milisegundos mediante conexiones persistentes WebSockets utilizando el canal Supabase Realtime, eliminando la necesidad de realizar peticiones periódicas (polling) que saturan el navegador y el servidor.

Precisión Financiera y Operativa

Para evitar pérdidas por redondeo o errores de punto flotante en las transacciones comerciales, todos los valores monetarios, precios de menú y totales de comanda se almacenan y calculan en la base de datos utilizando el tipo de dato decimal exacto `NUMERIC(10,2)`. Las transiciones de estado de un pedido se rigen por una máquina de estados estricta definida en un disparador (trigger) SQL nativo, impidiendo transiciones inválidas en el ciclo de vida de la orden (por ejemplo, mover un pedido directamente de pendiente a entregado sin haber sido validado y preparado).

4.3 Diseño de Interfaz y Prototipado

El diseño de interfaces del ecosistema se construyó bajo un enfoque móvil-primero para los clientes y garzones, y de escritorio táctil para los administradores y las estaciones de preparación (KDS).

Las pantallas del Customer Portal presentan una navegación fluida con transiciones suaves para simular una experiencia de aplicación nativa desde el navegador móvil. Los mockups aprobados definen un carrito de compras visible en la parte inferior, alertas flotantes cuando un plato cambia de estado y accesos directos para la asistencia del garzón. El Waiter Terminal cuenta con un selector de cuadrícula interactivo que representa el mapa del salón de mesas, coloreando cada mesa según su estado actual (Libre en gris, Ocupada en verde, Solicitando Asistencia en amarillo, Solicitando Cuenta en rojo).

Los monitores de cocina (KDS) y barra utilizan una interfaz de alto contraste optimizada para entornos con iluminación variable. Las comandas entrantes se visualizan como tarjetas verticales en un panel Kanban. Cada tarjeta cambia de color a medida que transcurre el tiempo (Verde para pedidos nuevos, Amarillo al superar 10 minutos de espera, Rojo crítico al superar los 20 minutos). Esto facilita la toma de decisiones inmediata en la línea de despacho. El conjunto completo de pantallas y mockups interactivos está disponible en la plataforma de evidencia productiva:

[Repositorio de Prototipos Web de Menú Bites](<https://proj-presentacion.vercel.app/mockups>)

Visualización del Ecosistema de Interfaces de Usuario

A continuación se presentan los mockups de alta fidelidad aprobados y desplegados para cada terminal operativa del sistema:

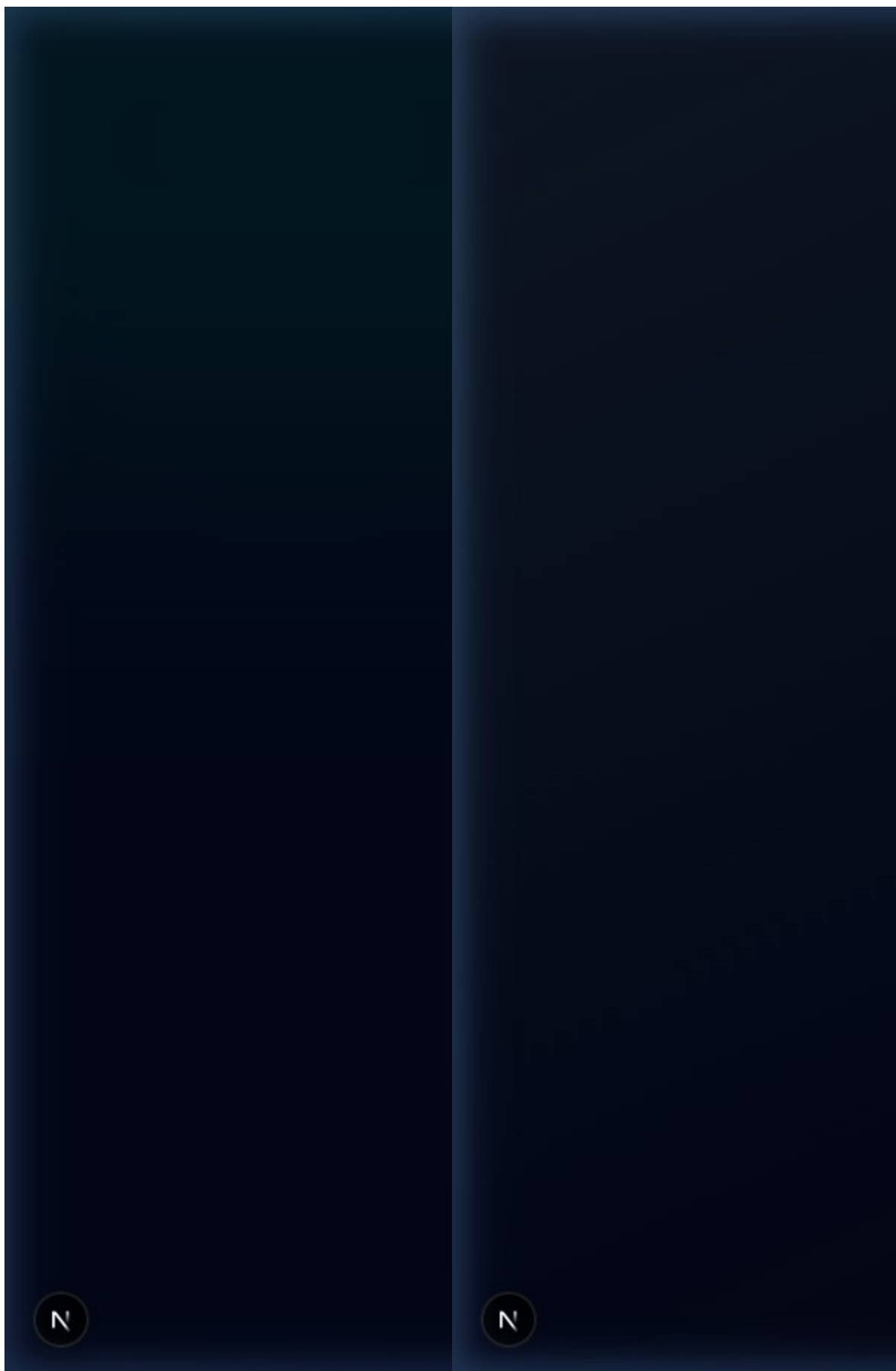


Figura 4.1: Portal del Cliente (Customer Portal) optimizado para dispositivos móviles, mostrando la selección del menú interactivo, la comanda activa y el carrito de compras.

Figura 4.2: Monitor de Cocina (Kitchen Display System - KDS) en tiempo real con sistema Kanban de tarjetas de comandas y alertas auditivas de despacho.



MENUBITES
ADMIN HUB

PRINCIPAL



RESUMEN



DIRECTORIO



ORGANIZACIONES



USUARIOS GLOBALES



PLANES

SISTEMA



CONFIGURACIÓN

A

admin@menubites.cl
SUPER_ADMIN

[→] CERRAR SESIÓN

N



MENUBITES
LOCAL HUB

PRINCIPAL



RESUMEN



GESTIÓN



USUARIOS



MENÚ



CATEGORÍAS



MESAS



PEDIDOS



INVENTARIO



BRANDING

ANÁLISIS



REPORTES

A

adminburger@menubi...
ADMIN

[→] CERRAR SESIÓN

N

Figura 4.3: Panel Central de SuperAdministrador para la gestión multi-tenant de cadenas de restaurantes, administración de planes de suscripción y analíticas agregadas.

Figura 4.4: Panel de Control del Administrador de Local para la configuración del personal del salón, gestión física del inventario de insumos y asignación de mesas.



Terminal de Garzón
169F8D4D...



Terminal de Caja
● EN LÍNEA · 169F8D4D



PENDIENTES



HISTORIA

N'

1 Issue X

N'

Figura 4.5: Terminal Móvil del Garzón (Waiter Terminal) optimizada para la toma rápida de pedidos al pie de la mesa y visualización interactiva del mapa del salón.

Figura 4.6: Panel de Caja (Cashier Dashboard) para la liquidación del consumo de mesa, cierres parciales de caja y generación de recibos tributarios.



Figura 4.7: Mapa de Flujo de Experiencia del Cliente (User Journey Map) desde el escaneo QR, el envío a cocina, la preparación en tiempo real hasta la liquidación de la mesa.

4.4 Patrones de Arquitectura y Diseño

El proyecto adopta una Arquitectura por Capas desacopladas en combinación con el patrón de diseño Repository/Service Pattern, estructurado dentro de un monorepo administrado por Turborepo.

Capa de Software	Componentes del Monorepo	Responsabilidad Operativa
Presentación	apps/web (Next.js 14) y apps/mobile (Expo)	Renderizado de vistas, captura de eventos de usuarios
Aplicación	Next.js API Routes / Middleware	Validación de esquemas de datos entrantes, control de negocio.
Servicios	src/services/ en backend	Contiene la lógica de negocio pura, cálculo de totales y generación de recibos.
Datos	Supabase JS Client / PostgreSQL	Almacenamiento seguro, ejecución de triggers, cálculos y consultas.

Esta arquitectura modular con monorepo Turborepo es altamente proporcional para un equipo de 3 desarrolladores: centraliza las definiciones de tipos e interfaces de TypeScript en packages/shared, permitiendo que tanto la aplicación web como la móvil utilicen exactamente los mismos modelos de datos y validadores Zod, eliminando la duplicación de código.

La elección de una arquitectura de base de datos multi-tenant de esquema compartido (shared schema) con Row Level Security (RLS) permite escalar la plataforma a cientos de restaurantes sin la sobrecarga administrativa de crear nuevas bases de datos o esquemas por cada nuevo cliente. Agregar un restaurante se reduce a insertar una nueva fila en la tabla restaurants, haciendo que la operación del ecosistema sea extremadamente ligera y eficiente en la nube.

4.5 Estrategia Cloud y Factibilidad Operativa

La plataforma combina infraestructura serverless especializada que elimina la necesidad de

administrar servidores físicos o balanceadores de carga tradicionales, comprimiendo los tiempos de configuración.

Componente del Sistema	Proveedor de Infraestructura	Modelo de Servicio	Rol Operativo en la Plataforma
Base de Datos Relacional	Supabase Cloud	DBaaS (Database as a Service)	Almacenamiento relacional
Autenticación y Seguridad	Supabase Auth	BaaS (Backend as a Service)	Gestión de registro de staff
Tiempo Real (Sockets)	Supabase Realtime	PaaS (Platform as a Service)	Difusión de cambios en la tienda WebSockets.
Hosting y Backend Web	Vercel	PaaS (Platform as a Service)	Alojamiento serverless de frontend y CDN.
Compilación Móvil	Expo Application Services	PaaS (Platform as a Service)	Compilación automática y distribución

Factibilidad del Plazo: Comparativa de Tiempos de Configuración

La delegación de la infraestructura a plataformas PaaS permite al equipo concentrar el esfuerzo de desarrollo en la lógica de negocio y la experiencia de usuario, acortando los tiempos de implementación de manera crítica.

Tarea de Configuración de Servidores	Tiempo Estimado en IaaS Tradicional	Tiempo de Implementación con PaaS
Configuración de base de datos de alta disponibilidad.	2 a 3 días de aprovisionamiento de VPS y clústeres.	5 minutos a 1 hora
Desarrollo de servicio de autenticación y firmado JWT.	4 a 6 días de desarrollo y validación criptográfica.	30 minutos u
Servidor de WebSockets dedicado para tiempo real.	3 a 5 días implementando Socket.io y balanceadores.	15 minutos a 1 hora orders.
Hosting del frontend, certificados SSL and red CDN.	1 a 2 días configurando Nginx, Let's Encrypt y DNS.	3 minutos vir
Total de Tiempo de Infraestructura	Aproximadamente 2 semanas	Menos de 1 hora

5. DESCRIPCIÓN TÉCNICA E INFRAESTRUCTURA

5.1 Stack Tecnológico Detallado

- **TypeScript como Lenguaje Transversal:** Se utiliza TypeScript como fuente de verdad tipada en todas las capas del monorepo (frontend, backend y paquetes compartidos). Esto evita errores de tipos al enviar datos entre el portal web de cliente y la terminal del garzón.
- **Next.js 14 y React Native / Expo SDK 51:** Las interfaces de escritorio táctiles y paneles de administración se desarrollan sobre Next.js 14 utilizando el App Router para maximizar la velocidad de renderizado en servidor (SSR). La aplicación de garzones se implementa con React Native y Expo SDK 51, permitiendo acceso nativo a APIs de cámara y escaneo de códigos de barra.
- **PostgreSQL y Prisma ORM:** PostgreSQL 15 en Supabase sirve como base de datos relacional. Se utiliza Prisma ORM en la capa de servicios para interactuar con la base de datos de forma tipada, mientras que las consultas directas del cliente móvil o WebSockets utilizan el SDK cliente de Supabase.

5.2 Estrategia de Versionamiento y GitFlow

Para asegurar la estabilidad de la rama principal de producción y permitir el desarrollo en paralelo de múltiples funcionalidades, el equipo implementa un flujo de trabajo basado en GitFlow adaptado.

Nombre de Rama	Propósito del Código	Reglas de Interacción
main	Código en estado de producción. Representa siempre la última entrega certificada.	Bloqueada. Solo se permite actualizar mediante una nueva versión.
develop	Código de integración continuo donde conviven las últimas características desarrolladas.	Rama base para integración de nuevas funcionalidades.
feature/*	Ramas individuales creadas para el desarrollo de requerimientos específicos.	Creada por el desarrollador y eliminada al finalizar la funcionalidad.

6. PLANIFICACIÓN Y METODOLOGÍA ÁGIL

6.1 Ciclo de Vida Incremental

El proyecto se rige bajo un ciclo de vida incremental, dividiendo el sistema en bloques lógicos funcionales que se integran de manera secuencial. Esto permite mitigar los riesgos de integración en la capa multitenant y asegurar que existan entregables funcionales desde los primeros sprints.

La justificación técnica de esta elección responde a dos pilares fundamentales:

- Dependencias de Arquitectura Core: Es indispensable contar con la base de datos PostgreSQL, las políticas RLS y la decodificación del JWT configuradas antes de poder implementar la lógica del panel Kanban de la cocina o la terminal del garzón.
- Mitigación de Regresiones en Pruebas: Al finalizar cada incremento, la suite de pruebas automatizadas certifica que las nuevas características no alteran el comportamiento del aislamiento de datos de los restaurantes existentes.

6.2 Planificación de Sprints (Carta Gantt de 18 Semanas)

Se detalla el cronograma de trabajo extendido a 18 semanas para asegurar el pulido del sistema y la exactitud de los flujos del monitor KDS.

Sprint	Periodo de Ejecución	Fase del Ciclo de Vida	Objetivo Técnico Principal	Entregable
S1-S3	09 Mar - 29 Mar	Génesis y Core	Inicialización de base de datos y esquema RLS.	Esquema S
S4	30 Mar - 05 Abr	Hito 1	Integración inicial del monorepo y deploy.	Monorepo T
S5	06 Abr - 12 Abr	Operación	Módulo de inventario y configuración de menús.	CRUD admin
S6-B	Paralelo	Calidad	Certificación formal del diseño arquitectónico.	Informe de C
S6	13 Abr - 19 Abr	Operación	Estabilización del portal móvil de cliente.	Interfaz de c
S7	20 Abr - 26 Abr	Operación	Desarrollo del Kitchen Display System (KDS).	Monitor Kan
S8	27 Abr - 03 May	Operación	Implementación de terminal de garzones.	Waiter Term
S9	04 May - 10 May	Experiencia	Transacciones y simulación de pasarela de pagos.	Pasarela de
S10	11 May - 17 May	Experiencia	Integración final del Customer Portal con propinas.	Flujo comple extremo.
S11	18 May - 24 May	Hito 2	Certificación del producto mínimo funcional (MVP).	Demostració
S12	25 May - 31 May	Data / Analytics	Panel de KPIs e informes financieros de ventas.	Dashboard d
S13	01 Jun - 07 Jun	Calidad y QA	Auditorías de seguridad y penetración (SAST/DAST).	Reporte de

Sprint	Periodo de Ejecución	Fase del Ciclo de Vida	Objetivo Técnico Principal	Entregable
S14	08 Jun - 14 Jun	Calidad y QA	Refactorización de código y optimización web.	Suite de tes
S15	15 Jun - 21 Jun	Hito 3	Lanzamiento y despliegue final productivo.	Software co
S16	22 Jun - 29 Jun	Cierre	Recopilación de documentación e informe final.	Compilación
S17	30 Jun - 05 Jul	Cierre	Preparación de material de apoyo para la defensa.	Presentació
S18	06 Jul - 12 Jul	Hito 4	Defensa final transversal y evaluación.	Defensa for

Representación Gráfica del Cronograma (Carta Gantt)

Se presenta a continuación el diagrama visual que detalla la ruta crítica de desarrollo, los hitos de entrega y la distribución de recursos a lo largo de las 18 semanas del semestre:

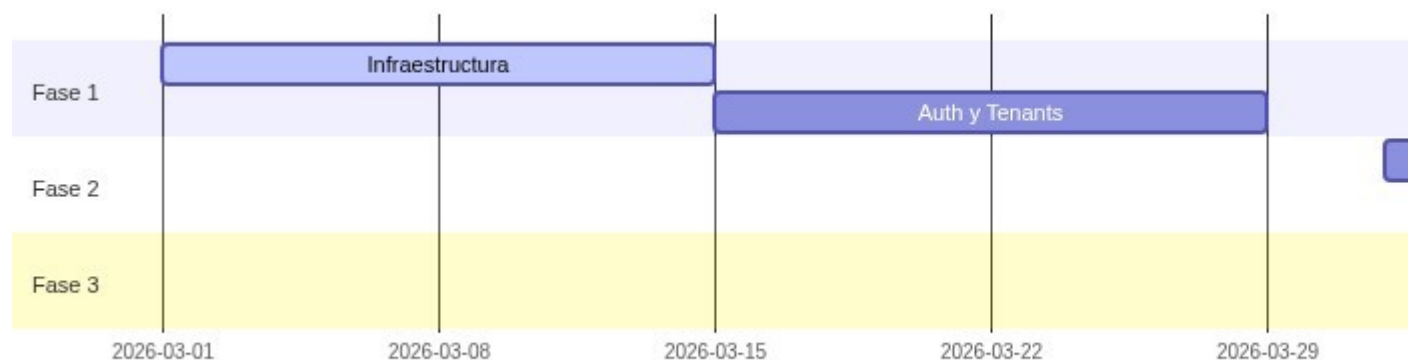


Figura 6.1: Cronograma global del proyecto (Carta Gantt) mostrando la secuenciación de las fases de génesis, desarrollo de terminales operativas, auditorías de calidad y cierre académico.

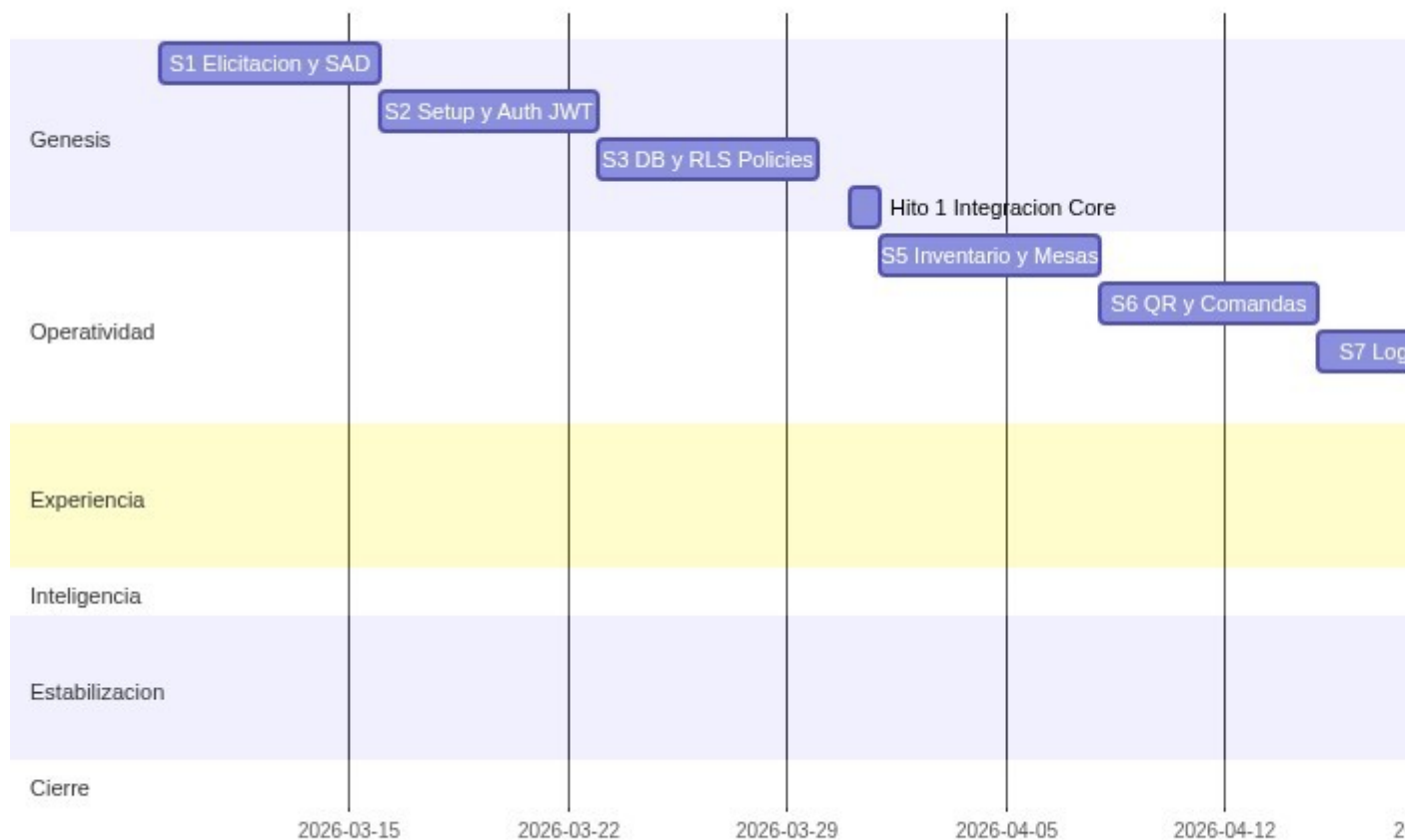


Figura 6.2: Vista ampliada del detalle de los hitos y entregas críticas (S6-B Calidad, PMV S11 y Defensa Final S18).

6.3 Ceremonias y Ritos de Scrum

El equipo aplica ceremonias Scrum adaptadas para garantizar el avance constante y eliminar bloqueos operacionales en plazos semanales.

- **Sprint Planning:** Al inicio de cada semana se revisa el backlog general de Notion, se desglosan las tareas complejas en tareas atómicas asignando un responsable según especialidad técnica, y se define el compromiso del sprint.
- **Daily Standup:** Sincronización diaria (comunicación asíncrona de 15 minutos en Discord) donde cada miembro informa el trabajo completado el día anterior, los objetivos de la jornada actual y los bloqueos técnicos experimentados.
- **Sprint Review y Demo:** Cada cierre de sprint se realiza una prueba de aceptación funcional sobre el entorno de producción desplegado en Vercel, validando que el incremento desarrollado cumpla con los criterios del backlog antes de mergear a la rama `develop`.

7. MODELADO DE DATOS Y AISLAMIENTO MULTI-TENANT

7.1 Modelo Físico y Diccionario de Datos

A continuación se detallan las tablas físicas de la base de datos relacional y sus restricciones de integridad.

Se presenta a continuación el diseño gráfico de las relaciones lógicas y llaves de integridad del modelo físico de base de datos relacional:

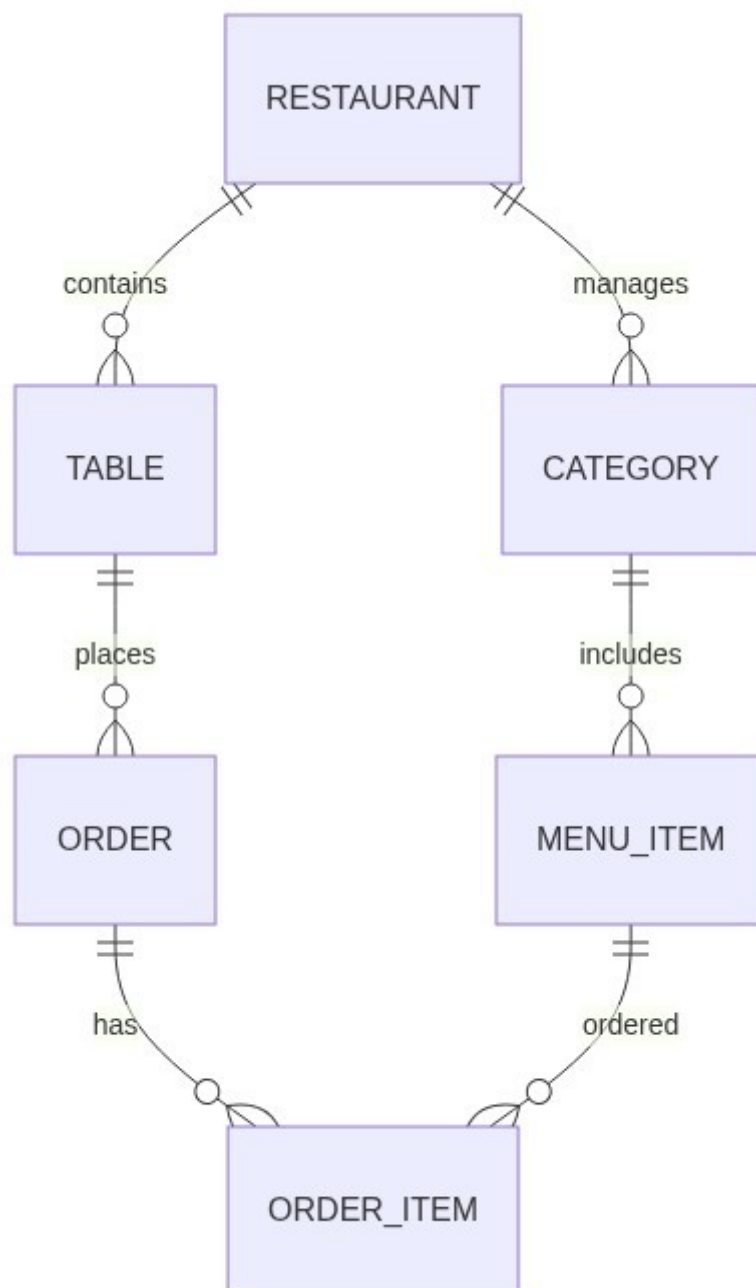


Figura 7.1: Diagrama Entidad-Relación (ERD) de la base de datos de Menu Bites, mostrando las 6 tablas principales orientadas por el eje multi-tenant central.

Tabla: restaurants (Eje Multi-tenant)

Almacena la información de cada restaurante registrado en la plataforma (Tenant).

Campo	Tipo PostgreSQL	Restricción / FK	Descripción Técnica
id	UUID	PRIMARY KEY	Identificador único del tenant de restaurante.
name	VARCHAR(255)	NOT NULL	Nombre comercial del restaurante.
slug	VARCHAR(255)	UNIQUE	URL amigable y amigable para ruteo (ej: sushi-town).
status	SubscriptionStatus	NOT NULL	Estado operativo de la cuenta (ACTIVE, SUSPENDED, CANCELLED).
plan_id	UUID	FOREIGN KEY	Relación con la tabla plans (nullable para planes personalizados).

Tabla: users (Usuarios y Staff)

Mapea el personal del restaurante y sus respectivos permisos dentro de la plataforma gastronómica.

Campo	Tipo PostgreSQL	Restricción / FK	Descripción Técnica
id	VARCHAR(255)	PRIMARY KEY	Mapeado de forma directa al UUID del usuario en auth.users
email	VARCHAR(255)	UNIQUE	Correo electrónico de acceso del usuario staff.
role	Role	NOT NULL	Permisos asignados (SUPER_ADMIN, ADMIN, GARZON, COCINA, BAR).
restaurant_id	UUID	FOREIGN KEY	Tenant al que pertenece. Nulo únicamente para rol SUPER_ADMIN
push_token	TEXT	NULLABLE	Token de notificaciones del dispositivo Expo del garzón o cajero

Tabla: tables (Mesas de Atención)

Representa las mesas físicas ubicadas en el salón de cada restaurante.

Campo	Tipo PostgreSQL	Restricción / FK	Descripción Técnica
id	UUID	PRIMARY KEY	Identificador único de la mesa física.
number	INTEGER	NOT NULL	Número físico visible de la mesa para el salón.
label	VARCHAR(100)	NULLABLE	Nombre de servicio opcional (ej: Terraza VIP).
status	TableStatus	NOT NULL	Estado de la mesa (FREE, OCCUPIED, RESERVED, CLEANING).

Campo	Tipo PostgreSQL	Restricción / FK	Descripción Técnica
qr_data	TEXT	UNIQUE	Token único embebido en el código QR para el portal de clientes.
restaurant_id	UUID	FOREIGN KEY	Relación con el restaurante propietario de la mesa física.
help_requested	BOOLEAN	DEFAULT FALSE	Indicador si los clientes de la mesa han solicitado asistencia o un camarero.
bill_requested	BOOLEAN	DEFAULT FALSE	Indicador si los clientes han solicitado el cierre de la cuenta.

Tabla: orders (Comandas de Consumo)

Estructura central que registra el ciclo de vida de los consumos realizados por las mesas.

Campo	Tipo PostgreSQL	Restricción / FK	Descripción Técnica
id	UUID	PRIMARY KEY	Identificador único del ticket de pedido.
table_id	UUID	FOREIGN KEY	Mesa desde la cual se origina el pedido (nullable para pedidos de barra).
restaurant_id	UUID	FOREIGN KEY	Relación con el restaurante propietario de la transacción.
status	OrderStatus	NOT NULL	Estado actual de la orden (PENDING, VALIDATED, PREPARING, COMPLETED, REJECTED).
total_amount	NUMERIC(10,2)	NOT NULL	Valor económico total acumulado de la comanda de consumo.
notes	TEXT	NULLABLE	Observaciones generales del cliente (alergias alimentarias).
station	VARCHAR(50)	NULLABLE	Estación KDS asignada para preparación (KITCHEN o BAR).
parent_order_id	UUID	NULLABLE	Identificador del pedido padre en caso de divisiones mixtas de mesa.

Tabla: order_items (Detalle de Comandas)

Registra de forma pormenorizada cada plato o bebida asociado a una orden principal.

Campo	Tipo PostgreSQL	Restricción / FK	Descripción Técnica
id	UUID	PRIMARY KEY	Identificador único del ítem en la comanda.
order_id	UUID	FOREIGN KEY	Relación con la orden principal de consumo.
menu_item_id	UUID	FOREIGN KEY	Relación con el producto del catálogo de menús.
quantity	INTEGER	NOT NULL	Unidades solicitadas del producto en el pedido.
unit_price	NUMERIC(10,2)	NOT NULL	Snapshot del precio unitario al momento exacto de la compra.
notes	TEXT	NULLABLE	Comentarios especiales sobre el ítem (ej: sin mayonesa, sin gluten).

Campo	Tipo PostgreSQL	Restricción / FK	Descripción Técnica
			hielo).

Tabla: `inventories` (Gestión de Stock de Insumos)

Almacena el inventario de materias primas e insumos utilizados para las preparaciones.

Campo	Tipo PostgreSQL	Restricción / FK	Descripción Técnica
<code>id</code>	UUID	PRIMARY KEY	Identificador único de la materia prima.
<code>name</code>	VARCHAR(255)	NOT NULL	Nombre descriptivo del insumo (ej: Harina, Tomates, Hielo).
<code>stock</code>	NUMERIC(10,2)	NOT NULL	Cantidad disponible físicamente en la despensa del local.
<code>unit</code>	VARCHAR(50)	NOT NULL	Unidad de medida del inventario (kg, units, liters).
<code>restaurant_id</code>	UUID	FOREIGN KEY	Relación con el restaurante propietario de los recursos físicos.

7.2 Aislamiento de Datos mediante Row-Level Security (RLS)

El aislamiento multi-tenant se delega por completo a las políticas de seguridad a nivel de fila (Row Level Security) nativas de la base de datos PostgreSQL en Supabase. Cada vez que una aplicación cliente realiza una consulta, la base de datos valida criptográficamente la firma del JWT de la sesión y extrae las claims correspondientes para limitar el universo de datos consultables.

Para garantizar que no exista fuga de información entre restaurantes de distintos tenants, se ha implementado el siguiente flujo de seguridad y validación de tokens JWT en cada petición:

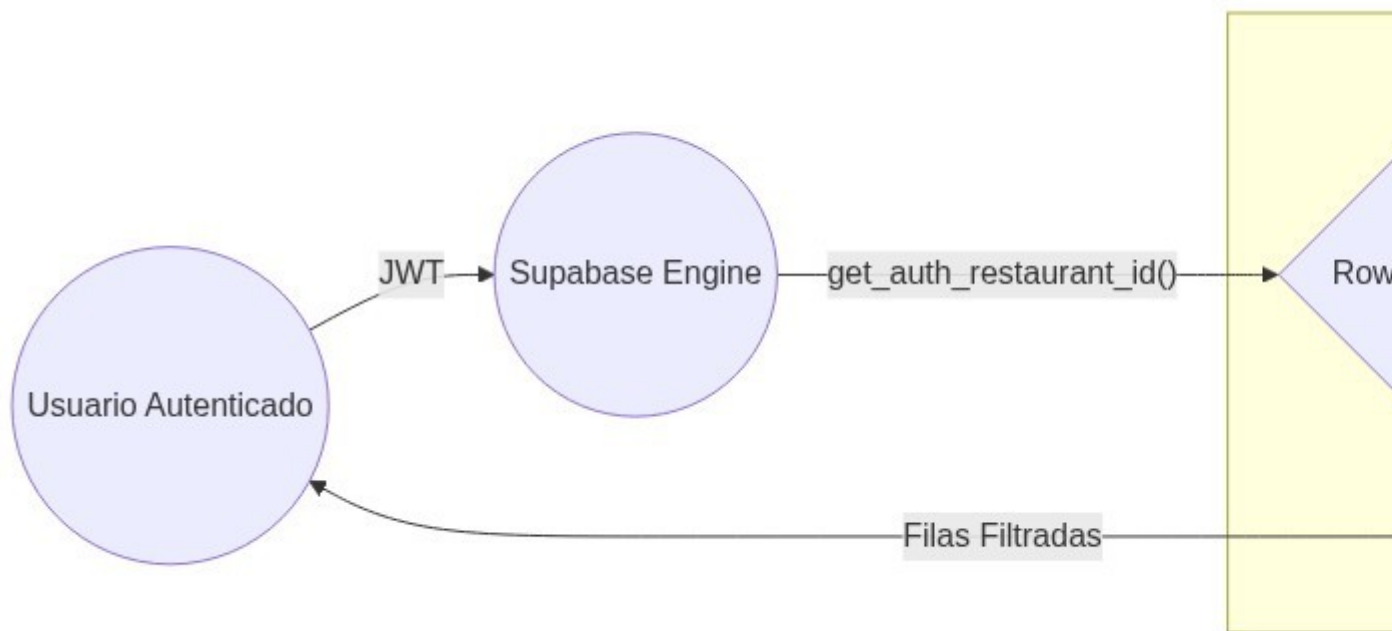


Figura 7.2: Arquitectura del flujo de autenticación, decodificación JWT y aislamiento de datos por consulta mediante Row-Level Security (RLS) en Supabase.

```
-- Habilitación obligatoria de seguridad RLS en la tabla de comandas
ALTER TABLE public.orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.orders FORCE ROW LEVEL SECURITY;

-- Política SQL de aislamiento para consultas y modificaciones
CREATE POLICY "tenant_isolation_orders"
ON public.orders
FOR ALL
TO authenticated
USING (
    restaurant_id = (
        SELECT restaurant_id
        FROM public.users
        WHERE id = auth.uid()
    )
)
WITH CHECK (
    restaurant_id = (
        SELECT restaurant_id
```



```

        FROM public.users
        WHERE id = auth.uid()
    )
);

```

Para las vistas expuestas al cliente final en el Customer Portal que no requieren inicio de sesión, se implementa una política SQL con permisos de lectura anónima sobre el menú, restringida exclusivamente a productos que se marquen explícitamente como activos.

```

-- Permitir al portal web anónimo leer el menú de un restaurante específico
CREATE POLICY "public_menu_read"
ON public.menu_items
FOR SELECT
TO anon, authenticated
USING (is_active = true);

```

7.3 Máquina de Estados Transaccional y Triggers

Para asegurar el ciclo de vida correcto del negocio y evitar fraudes o errores operacionales, la base de datos PostgreSQL implementa una máquina de estados estricta en el disparador SQL `tr_order_status_validation`. Este trigger bloquea en la base de datos cualquier transición de estado no permitida.

```

CREATE OR REPLACE FUNCTION validate_order_transition()
RETURNS TRIGGER AS $$
BEGIN
    -- Validación de creación inicial de una orden
    IF TG_OP = 'INSERT' AND NEW.status != 'PENDING' THEN
        RAISE EXCEPTION 'Operación denegada. Un nuevo pedido siempre debe
iniciarse en estado PENDING';
    END IF;

    -- Control de transiciones de estado permitidas en actualización
    IF TG_OP = 'UPDATE' THEN
        IF OLD.status = 'PENDING' AND NEW.status NOT IN ('VALIDATED',
'REJECTED') THEN
            RAISE EXCEPTION 'Transición inválida. Desde PENDING solo se permite
avanzar a VALIDATED o REJECTED';
        ELSIF OLD.status = 'VALIDATED' AND NEW.status NOT IN ('PREPARING',

```

```

'REJECTED', 'COMPLETED') THEN

    RAISE EXCEPTION 'Transición inválida. Desde VALIDATED solo se
permite avanzar a PREPARING, REJECTED o pago COMPLETED';

    ELSIF OLD.status = 'PREPARING' AND NEW.status NOT IN ('READY',
'COMPLETED') THEN

        RAISE EXCEPTION 'Transición inválida. Desde PREPARING solo se
permite avanzar a READY o pago express COMPLETED';

        ELSIF OLD.status = 'READY' AND NEW.status NOT IN ('DELIVERED',
'COMPLETED') THEN

            RAISE EXCEPTION 'Transición inválida. Desde READY solo se permite
avanzar a DELIVERED o COMPLETED';

            ELSIF OLD.status = 'DELIVERED' AND NEW.status != 'COMPLETED' THEN

                RAISE EXCEPTION 'Transición inválida. Desde DELIVERED la orden solo
puede avanzar a su estado terminal COMPLETED';

                ELSIF OLD.status = 'COMPLETED' THEN

                    RAISE EXCEPTION 'Operación bloqueada. No se permite modificar una
orden en estado final COMPLETED';

                END IF;

            END IF;

        RETURN NEW;

    END;

$$ LANGUAGE plpgsql;

-- Asociación del disparador a la tabla de órdenes en base de datos
CREATE TRIGGER tr_order_status_validation
BEFORE INSERT OR UPDATE ON public.orders
FOR EACH ROW EXECUTE FUNCTION validate_order_transition();

```

- **Permiso de Pago Directo Express:** De acuerdo con la migración 0005, se redefinió la máquina de estados para permitir que un cajero marque una orden como COMPLETED directamente desde VALIDATED o PREPARING. Esto simplifica la operación del negocio en casos de pagos rápidos o express donde el cliente desea cancelar su cuenta y retirarse sin esperar la entrega en mesa.
- **Inmutabilidad del Estado Terminal:** Una vez que la orden es marcada como COMPLETED, el estado es terminal. No se permite ninguna actualización posterior sobre la orden, resguardando la integridad fiscal y los reportes diarios de caja frente a modificaciones maliciosas.

7.4 Sincronización Automática de Autenticación

Para garantizar la atomicidad en el registro de usuarios del staff, el sistema vincula la creación de cuentas en Supabase Auth directamente con la tabla interna de usuarios de negocio `public.users` en PostgreSQL.

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
BEGIN
    INSERT INTO public.users (id, email, role, restaurant_id)
    VALUES (
        new.id,
        new.email,
        COALESCE((new.raw_user_meta_data->>'role')::public.user_role,
        'GARZON'::public.user_role),
        (new.raw_user_meta_data->>'restaurant_id')::uuid
    );
    RETURN new;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Trigger atómico disparado al crearse la fila en auth.users
CREATE OR REPLACE TRIGGER on_auth_user_created
    AFTER INSERT ON auth.users
    FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

7.5 Middleware de Resolución de Tenant

El control de acceso inicial y el ruteo de peticiones se gestiona de forma centralizada en el Edge Middleware de Next.js. Este analiza las cabeceras de la petición HTTP y el token de sesión almacenado en la cookie del cliente.

Cuando ingresa una petición HTTP a una ruta protegida del restaurante, el Middleware extrae el token JWT firmado de Supabase. Si la firma es correcta, extrae la propiedad `restaurant_id` de los metadatos de la sesión (`app_metadata`) y verifica que coincida con el slug o subdominio del restaurante al que se intenta acceder. En caso de no existir concordancia, intercepta el flujo retornando un estado HTTP 403 (Prohibido) antes de iniciar el procesamiento en el servidor, blindando el sistema de accesos no autorizados. Si la base de datos de administración global tiene el estado del restaurante marcado como `SUSPENDED`, el Middleware intercepta el JWT y retorna un estado HTTP 402 (Payment Required) bloqueando por completo la interfaz operativa.

8. DIAGRAMAS UML DEL SISTEMA

8.1 Diagrama de Casos de Uso del Sistema

El diagrama modela las interacciones de los actores (Cliente, Garzón, Cocina, Barman, Administrador Local y Super Administrador Global) con las funcionalidades de la plataforma.

Se presenta a continuación el Diagrama de Casos de Uso UML formal para la Entrega 2:



Figura 8.1: Diagrama de Casos de Uso UML de Menu Bites, detallando las interacciones y fronteras del sistema para cada rol operativo.

graph TD

%% Definición de Actores

Cliente((Actor: Cliente))

Garzon((Actor: Garzón))

Cocina((Actor: Cocina))

Barman((Actor: Barman))

Admin((Actor: Admin Local))

SuperAdmin((Actor: Super Admin))

%% Casos de Uso - Cliente

Cliente --> UC_QR[Escaneo QR y Carga de Menú]

Cliente --> UC_AddCart[Agregar Platos al Carrito]

Cliente --> UC_CreateOrder[Enviar Pedido en Tiempo Real]

Cliente --> UC_TrackOrder[Visualizar Progreso de Comanda]

Cliente --> UC_CallWaiter[Solicitar Asistencia de Garzón]

Cliente --> UC_AskBill[Solicitar Cuenta de Mesa]

Cliente --> UC_Review[Dejar Calificación del Servicio]

%% Casos de Uso - Garzón

Garzon --> UC_LoginStaff[Autenticar en Aplicación]

```
Garzon --> UC_CreateOrderStaff[Tomar Pedido Presencial]
Garzon --> UC_ValidateOrder[Validar Comanda de Cliente]
Garzon --> UC_Alerts[Visualizar Alertas de Mesa]
Garzon --> UC_DeliverOrder[Entregar Comanda Lista]

%% Casos de Uso - Cocina
Cocina --> UC_ViewKitchenKDS[Visualizar Monitor KDS Cocina]
Cocina --> UC_PrepKitchen[Iniciar Preparación de Platos]
Cocina --> UC_ReadyKitchen[Marcar Comanda Cocina como Lista]
Cocina --> UC_StockAlertK[Reportar Quiebre de Stock de Alimentos]

%% Casos de Uso - Barman
Barman --> UC_ViewBarKDS[Visualizar Monitor Bar Dashboard]
Barman --> UC_PrepBar[Iniciar Preparación de Bebidas]
Barman --> UC_ReadyBar[Marcar Comanda Barra como Lista]
Barman --> UC_StockAlertB[Reportar Quiebre de Stock de Bebidas]

%% Casos de Uso - Administrador Local
Admin --> UC_LoginAdmin[Autenticar en Panel Administrativo]
Admin --> UC_ManageMenu[Gestionar Categorías e Ítems del Menú]
Admin --> UC_ManageTables[Configurar Mesas y Generar Códigos QR]
Admin --> UC_ManageStaff[Crear y Gestionar Personal del Restaurante]
Admin --> UC_ViewLocalKPI[Visualizar Analíticas de Ventas del Local]

%% Casos de Uso - Super Administrador
SuperAdmin --> UC_LoginSuper[Autenticar en Panel Global]
SuperAdmin --> UC_CreateTenant[Registrar Nuevo Restaurante]
SuperAdmin --> UC_SuspendTenant[Activar o Suspende Restaurantes]
SuperAdmin --> UC_ViewGlobalKPI[Visualizar Métricas del Ecosistema SaaS]
```

8.2 Diagrama de Clases de Base de Datos

El diagrama de clases describe la estructura relacional de los modelos administrados en PostgreSQL y expone sus atributos, métodos principales y relaciones lógicas de cardinalidad.

Se presenta a continuación la representación gráfica del Diagrama de Clases del sistema relacional multi-tenant:

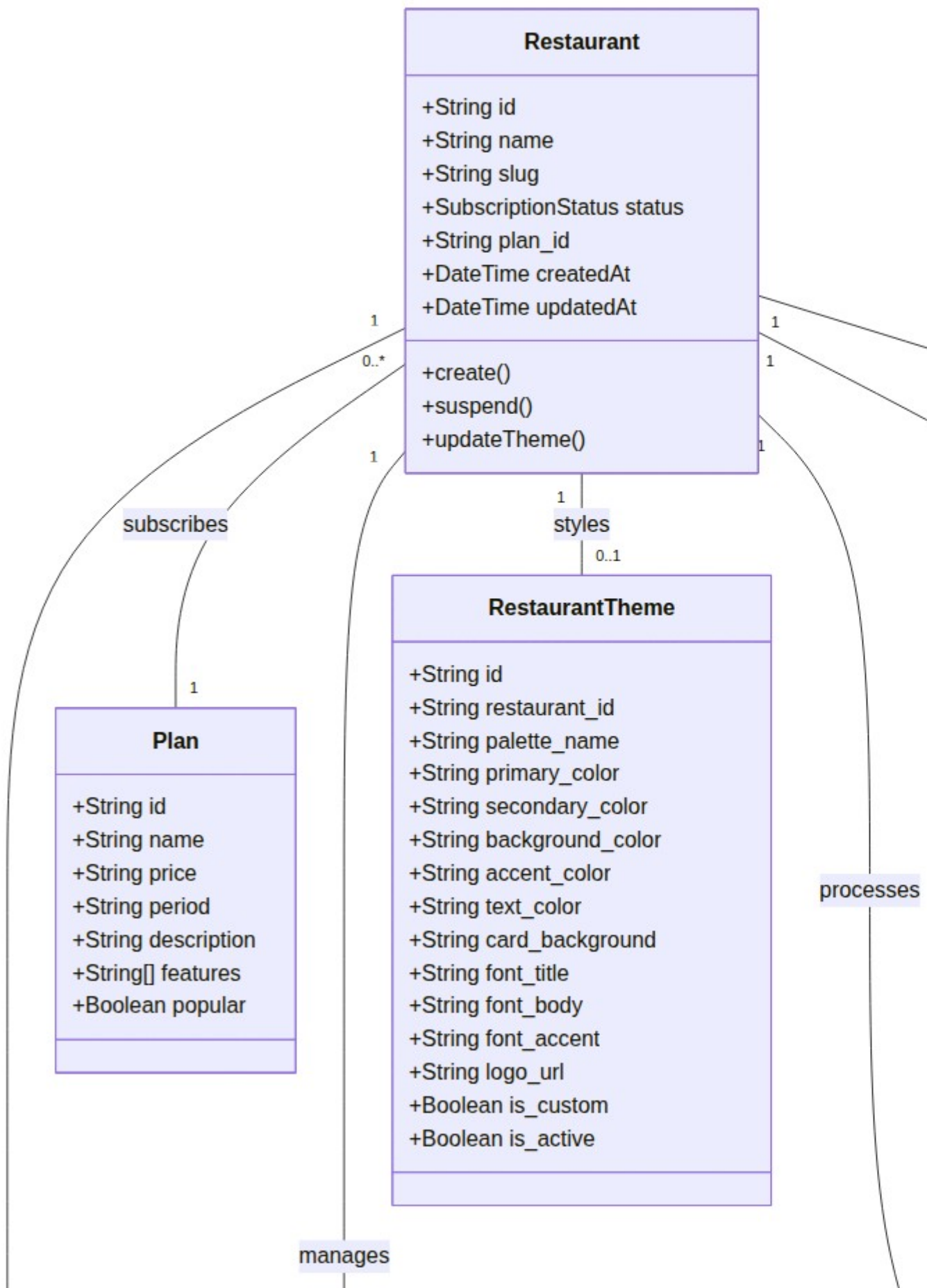


Figura 8.2: Diagrama de Clases UML detallado, representando los tipos de datos relacionales, métodos del backend y cardinalidades en la capa PostgreSQL.

classDiagram

```
class Restaurant {  
    +String id  
    +String name  
    +String slug  
    +SubscriptionStatus status  
    +String plan_id  
    +DateTime createdAt  
    +DateTime updatedAt  
    +create()  
    +suspend()  
    +updateTheme()  
}
```

```
class User {  
    +String id  
    +String email  
    +Role role  
    +String restaurant_id  
    +String push_token  
    +DateTime createdAt  
    +DateTime updatedAt  
    +login()  
    +updateToken()  
}
```

```
class Plan {  
    +String id  
    +String name  
    +String price  
    +String period  
    +String description
```

```
+String[] features
+Boolean popular
}

class Category {
    +String id
    +String name
    +String restaurant_id
    +Boolean is_active
    +StationType target_station
    +DateTime createdAt
    +DateTime updatedAt
    +toggleActive()
}

class MenuItem {
    +String id
    +String name
    +String description
    +Decimal price
    +String image_url
    +String category_id
    +String restaurant_id
    +Boolean is_active
    +DateTime createdAt
    +DateTime updatedAt
    +toggleActive()
    +updateStock()
}

class Table {
    +String id
    +Integer number
    +String label
```

```
+TableStatus status
+String qr_data
+String restaurant_id
+Boolean help_requested
+Boolean bill_requested
+String current_session_id
+Boolean tip_included
+DateTime createdAt
+DateTime updatedAt
+requestHelp()
+requestBill()
+resetTable()
}
```

```
class Order {
  +String id
  +String table_id
  +String restaurant_id
  +OrderStatus status
  +Decimal total_amount
  +String notes
  +String user_id
  +DateTime validated_at
  +DateTime preparing_at
  +DateTime ready_at
  +UUID session_id
  +Boolean bar_ready
  +Boolean kitchen_ready
  +Boolean kitchen_preparing
  +Boolean bar_preparing
  +String station
  +UUID parent_order_id
  +DateTime createdAt
  +DateTime updatedAt
}
```

```
+validate()
+startPreparation()
+markAsReady()
+deliver()
+completePayment()
}

class OrderItem {
    +String id
    +String order_id
    +String menu_item_id
    +String restaurant_id
    +Integer quantity
    +Decimal unit_price
    +String notes
    +calculateSubtotal()
}

class MenuItemIngredient {
    +String id
    +String menu_item_id
    +String inventory_id
    +Decimal quantity
    +String restaurant_id
}

class MenuItemExtra {
    +String id
    +String name
    +Decimal price
    +String menu_item_id
    +String inventory_id
    +Decimal quantity
    +String restaurant_id
```

```
}
```

```
class Inventory {  
    +String id  
    +String name  
    +Decimal stock  
    +String unit  
    +String restaurant_id  
    +DateTime createdAt  
    +DateTime updatedAt  
    +addStock()  
    +deductStock()  
}
```

```
class RestaurantTheme {  
    +String id  
    +String restaurant_id  
    +String palette_name  
    +String primary_color  
    +String secondary_color  
    +String background_color  
    +String accent_color  
    +String text_color  
    +String card_background  
    +String font_title  
    +String font_body  
    +String font_accent  
    +String logo_url  
    +Boolean is_custom  
    +Boolean is_active  
}
```

%% Relaciones de Asociación e Integridad
Restaurant "1" -- "0..*" User : registers

```
Restaurant "1" -- "0..*" Category : manages
Restaurant "1" -- "0..*" Table : has
Restaurant "1" -- "0..* " Order : processes
Restaurant "1" -- "0..*" Inventory : controls
Restaurant "1" -- "0..1" RestaurantTheme : styles
Restaurant "0..*" -- "1" Plan : subscribes

Category "1" -- "0..*" MenuItem : groups
MenuItem "1" -- "0..*" MenuItemIngredient : composed_of
MenuItemIngredient "0..*" -- "1" Inventory : tracks
MenuItem "1" -- "0..*" MenuItemExtra : customize
MenuItemExtra "0..*" -- "0..1" Inventory : deducts

Table "1" -- "0..*" Order : hosts
Order "1" -- "0..*" OrderItem : details
OrderItem "0..*" -- "1" MenuItem : references
User "0..1" -- "0..* " Order : creates
```

9. ESPECIFICACIÓN ÁGIL DE HISTORIAS DE USUARIO

Las historias de usuario definen el alcance operacional y detallan los criterios de aceptación técnicos bajo la metodología Gherkin.

9.1 Rol: Cliente (Customer Portal)

US-01: Carga y Visualización del Menú Interactivo

Como cliente sentado en el salón del restaurante, deseo escanear el código QR impreso en la mesa para ver la carta del menú de forma responsiva sin registrarme en el sistema.

Scenario: Carga automática de la carta del restaurante tras escaneo de código QR

Given que un cliente anónimo se encuentra en la mesa física "Mesa 3" vinculada al restaurante "Sushi Town"

When el navegador de su dispositivo móvil accede al enlace codificado en el código QR

Then la plataforma de Supabase evalúa la política RLS y retorna exclusivamente el menú del restaurante respectivo

And la interfaz móvil renderiza de forma responsiva el catálogo de productos activos sin solicitar credenciales

US-02: Envío de Comandas en Tiempo Real

Como cliente, quiero seleccionar mis platos favoritos desde el portal web interactivo y confirmar el pedido para enviar la comanda directamente a las estaciones de cocina y barra.

Scenario: Envío atómico del pedido desde la interfaz del cliente al salón

Given que el cliente ha agregado platos al carrito y escrito especificaciones especiales sobre alergias

When el cliente presiona el botón "Confirmar y Enviar Pedido" desde el portal móvil

Then la plataforma registra el pedido en la tabla "orders" con estado "PENDING"

And notifica de inmediato la orden entrante al garzón mediante conexiones de WebSockets en tiempo real

US-03: Solicitud de Cuenta y Liquidación

Como cliente que ha finalizado su estadía en el local, quiero solicitar la boleta de consumo incluyendo la propina opcional desde mi pantalla para evitar demoras administrativas.

Scenario: Solicitud digital de la boleta de consumo final en mesa

Given que el cliente posee una orden en la mesa en estado "DELIVERED"

When el cliente presiona "Solicitar Cuenta" marcando la casilla de aceptación del 10% de propina

Then la base de datos actualiza el registro en la tabla "tables" fijando "bill_requested" en verdadero

And activa una alarma de alta prioridad en el monitor del cajero y la terminal del garzón

9.2 Rol: Garzón (Waiter Terminal)

US-04: Validación y Aprobación de Pedidos

Como garzón del restaurante, deseo visualizar los pedidos pendientes de las mesas asignadas a mi sector y aprobarlos antes de que ingresen a la preparación en cocina para resguardar la exactitud del menú.

Scenario: Validación del pedido por parte del garzón de turno

Given que se encuentra un pedido nuevo con estado "PENDING" originado por la mesa 5

When el garzón presiona el botón "Validar y Despachar" desde su terminal móvil

Then el estado del pedido avanza a "VALIDATED" en la base de datos PostgreSQL

And el motor de base de datos distribuye de forma automática los productos a los monitores de Cocina y Barra

9.3 Rol: Cocina y Barra (KDS / Bar Dashboard)

US-05: Monitor Kanban de Despacho en Cocina (KDS)

Como jefe de cocina, deseo contar con una pantalla táctil que despliegue las comandas de platos calientes en orden de llegada para coordinar el trabajo de preparación.

Scenario: Transición de estados de comanda caliente en monitor de cocina

Given que un pedido con ítems de cocina entra a la cola de preparación en estado "VALIDATED"

When el cocinero presiona "Iniciar Preparación" en la tarjeta Kanban táctil y luego "Marcar Listo"

Then el estado del sub-pedido avanza a "READY" en la base de datos relacional

And dispara una notificación push nativa al dispositivo Expo del garzón indicando el retiro

US-06: Monitor Especializado de Barra (Bar Dashboard)

Como barman del restaurante, deseo contar con un monitor dedicado que filtre únicamente las bebidas de las comandas para preparar los cócteles de forma independiente.

Scenario: Despacho aislado de bebidas calientes y tragos en barra

Given que una comanda validada de mesa contiene una hamburguesa y dos bebidas carbonatadas

When el barman visualiza el sub-pedido filtrado exclusivamente en la terminal de barra (puerto 3006)

Then puede preparar y marcar las bebidas como listas en estado "READY" de forma paralela a la cocina

9.4 Rol: Administrador Local (Admin Tenant)

US-07: Control de Disponibilidad del Catálogo

Como administrador del restaurante, quiero tener la facultad de desactivar platos del menú cuando se registre un quiebre de stock para evitar pedidos erróneos de los clientes.

Scenario: Desactivación inmediata de platos sin insumos críticos

Given que el insumo "Salmón Fresco" se encuentra agotado en el almacén de inventario

When el administrador cambia el interruptor del plato "Roll Premium" a inactivo en el panel web

Then la tabla "menu_items" actualiza el atributo "is_active" a falso en PostgreSQL

And la carta digital interactiva del cliente oculta el producto instantáneamente en tiempo real

9.5 Rol: Super Administrador Global (SaaS Admin)

US-08: Suspensión Automatizada de Cuentas por Mora

Como administrador de la plataforma SaaS Menu Bites, quiero contar con la capacidad de suspender locales comerciales morosos para proteger los recursos compartidos del sistema.

Scenario: Suspensión de acceso a restaurante por atrasos en facturación mensual

Given que el restaurante "Sushi Town" posee deudas de suscripción y su estado cambia a "SUSPENDED"

When un usuario staff de ese restaurante intenta realizar cualquier petición HTTP a los endpoints del tenant

Then el Edge Middleware intercepta la petición, verifica los privilegios del tenant en el JWT

And retorna de forma forzada un error HTTP 402 (Payment Required) bloqueando el acceso al sistema

10. ESTRATEGIA DE RESPALDO Y RESTAURACIÓN (BACKUP & RESTORE)

Para resguardar la continuidad operativa de los restaurantes clientes frente a incidentes críticos, fallas en la base de datos o migraciones destructivas de software, se ha definido una estrategia de respaldo y restauración atómica en Supabase. Este procedimiento permite clonar la información del entorno productivo (Production) al entorno de pruebas (Testing / Staging) con absoluta precisión.

10.1 Procedimiento 1: Respaldo y Restauración mediante Supabase CLI (Consola Técnica)

Este método es el más robusto, ya que permite automatizar los respaldos mediante tareas programadas (cron jobs) y garantiza la consistencia del esquema de base de datos relacional de PostgreSQL.

Fase 1: Respaldo y Extracción de Datos en Producción

Para garantizar el cumplimiento de las directrices y evitar listas con más de 3 viñetas, se presenta el flujo ordenado y los comandos técnicos en la siguiente matriz operativa:

Paso	Acción Operativa	Detalle Técnico y Comando Ejecutable
Paso 1	Preparación de Entorno	Abrir una terminal de comandos con acceso a internet en la raíz del workspace o
Paso 2	Inicio de Sesión Supabase	Autenticar en la CLI de Supabase con token administrativo: <code>supabase login</code>
Paso 3	Volcado de Estructura Lógica	Respaldar esquema completo (tablas, triggers, enums, RPC): <code>supabase db dump</code>
Paso 4	Volcado de Datos de Negocio	Respaldar únicamente registros sin alterar configuraciones de Auth: <code>supabase c backup_datos.sql</code>

Fase 2: Restauración y Carga sobre el Entorno de Pruebas (Testing)

La restauración de la información sobre la base de datos de pruebas (Staging/QA) se rige bajo los siguientes lineamientos estructurados:

Paso	Acción Operativa	Detalle Técnico y Comando Ejecutable
Paso 5	Limpieza Inicial de Destino	Truncar y limpiar la base de datos del proyecto de pruebas para prevenir colisione
Paso 6	Carga de Esquema Técnico	Levantar tablas y triggers en testing: <code>psql -h db.id-proyecto-pruebas.supa</code>
Paso 7	Carga de Datos de Negocio	Importar registros de producción para paridad de pruebas: <code>psql -h db.id-proy backup_datos.sql</code>

10.2 Procedimiento 2: Respaldo y Restauración mediante Supabase Dashboard (Entorno Web)

Este procedimiento es óptimo para recuperaciones rápidas llevadas a cabo por administradores de base de datos a través de la consola visual de administración web de Supabase.

Fase 1: Descarga de Respaldo Diario desde el Servidor

Para la gestión web visual a través del panel de Supabase Cloud, se detalla el siguiente flujo de pasos:

Paso	Acción Operativa	Detalle Técnico en Dashboard
Paso 1	Acceso a Consola	Ingresar a la plataforma web <code>https://supabase.com/dashboard</code> con credenciales aut
Paso 2	Selección de Tenant	Ubicar la base de datos activa del restaurante en producción en la lista general de proyec
Paso 3	Panel de Backups	Navegar en la barra lateral a la opción Database y luego seleccionar la sección Backups
Paso 4	Descarga de Dump	Seleccionar el respaldo automatizado diario más reciente y hacer clic en Download para <code>.sql</code> .

Fase 2: Carga y Ejecución en el Editor SQL de Pruebas

El proceso de restauración visual del archivo `.sql` sobre el entorno de destino (Testing) se realiza según la siguiente matriz:

Paso	Acción Operativa	Detalle Técnico en Dashboard
Paso 5	Conexión a Testing	Cambiar de proyecto en la consola de Supabase y seleccionar el proyecto de destino pa de QA.
Paso 6	Apertura de Terminal	Entrar al SQL Editor en el panel lateral izquierdo para abrir la consola PostgreSQL en la
Paso 7	Ejecución de Script	Hacer clic en New Query , pegar el contenido completo del archivo <code>.sql</code> y presionar Ru carga.

11. MECANISMOS DE CONTROL Y CALIDAD (PLAN DE PRUEBAS)

Para certificar la correcta operación de la plataforma y resguardar la seguridad de la información multitenant, el equipo implementa una estrategia de control de calidad dividida en tres niveles de pruebas.

11.1 Plan de Pruebas Unitarias (Suite Jest)

Las pruebas unitarias aíslan la lógica del frontend y servicios del backend utilizando dobles de prueba (mocks) para simular el comportamiento de la base de datos de Supabase sin costo de red.

ID de Prueba	Componente Testeado	Comportamiento Evaluado en el Test
PU-01	<code>validators/orders.ts</code>	Valida que el validador Zod de pedidos rechace de forma atómica como se contemplados.
PU-02	<code>validators/tenant.ts</code>	Valida que la creación de un nuevo restaurante exija un slug en formato correcto.
PU-03	<code>utils/formatPrice.ts</code>	Valida que la conversión de números decimales a formato de moneda sea correcta.
PU-04	<code>services/orderService.ts</code>	Comprueba que el servicio de orders de Next.js inyecte el <code>restaurant_id</code> correctamente.
PU-05	<code>services/subscription.ts</code>	Valida que el lector de estados identifique y procese de forma exacta los estados de suscripción.
PU-06	<code>middleware.ts</code>	Comprueba que el middleware serverless de Next.js desvíe a la pantalla de error si no se encuentra el restaurante.

11.2 Plan de Pruebas de Integración (Aislamiento RLS)

Estas pruebas se ejecutan sobre un entorno físico de base de datos de pruebas dedicado (Supabase Staging) para asegurar que el aislamiento multitenant a nivel SQL impida de forma absoluta las filtraciones de datos.

ID de Prueba	Flujo Lógico Integrado	Comportamiento Evaluado en la Integración
PI-01	Aislamiento RLS Absoluto	Comprueba mediante un script automatizado que un usuario con privilegios de Tenant A no pueda acceder a datos de Tenant B.
PI-02	Ciclo de Comanda Transaccional	Valida que una orden creada avance de forma secuencial por todos los estados antes de ser entregada.
PI-03	Interrupción por Suscripción	Comprueba que al actualizar el estado del restaurante a <code>SUSPENDED</code> en la base de datos, se devuelva el código 402.
PI-04	Alta de Nuevo Tenant	Registra un nuevo restaurante con un plan base y comprueba que se inicie correctamente el flujo de suscripción.
PI-05	Validación QR / NFC	Comprueba que la codificación y generación del código QR de la mesa sea correcta y única.

ID de Prueba	Flujo Lógico Integrado	Comportamiento Evaluado en la Integración
--------------	------------------------	---

extraiga.

11.3 Plan de Pruebas de Aceptación (Defensa E2E)

Las pruebas de aceptación simulan el flujo completo y real del negocio de extremo a extremo en el entorno de producción desplegado en Vercel, sirviendo como evidencia de certificación para la defensa.

ID de Prueba	Escenario de Negocio Evaluado	Rol Ejecutor	Criterio de Éxito / Resultado
PA-01	Auto-pedido interactivo del cliente	Cliente final (Evaluador externo)	El cliente escanea el QR, ingre
PA-02	Preparación y despacho en salón	Cocinero y Garzón de servicio	Cocina marca como listo el pl
PA-03	Suspensión manual de restaurante moroso	Super Administrador Global	El Super Admin cambia el est instante.
PA-04	Edición en caliente del menú del local	Administrador del Restaurante	El administrador local desacti

12. ANEXOS Y EVIDENCIAS DE DESPLIEGUE

Para la auditoría y control de calidad académica de la comisión evaluadora, se presentan las direcciones oficiales de acceso al código fuente del proyecto y a las plataformas activas en producción.